ORBITL1-AX25

PHASE 1 TECHNICAL REPORT | AX.25 FRAME CORE & PROTOCOL SPECIFICATION

Phase 1 COMPLETE	29 / 29 Tests PASS	0x906E CRC-16	0 – 4078 B Capacity
Software Baseline Gate Status	AX.25 Frame Core Suite	ISO/IEC 3309 Vector Match	Information Field Sizing

Phase 1 Architectural Scope & Objectives

Phase 1 establishes the fundamental Data Link Layer (Layer 2) data abstractions for the ORBITL-1 CubeSat communication system. The implementation provides strict Unnumbered Information (UI) frame assembly and disassembly adhering to the amateur AX.25 Version 2.2 protocol specification. Core capabilities include canonical callsign and SSID parsing, wire-format address octet shifting, control field validation (0x03 UI), Layer 3 protocol identification (0xF0 No Layer 3), and standard Frame Check Sequence (FCS) calculation using CRC-16/X-25 in little-endian order.

Phase 1 Automated Test Suite Breakdown (tests/test_ax25_core.py)

Test Class	Test Count	Functional Focus	Key Assertions & Verification Conditions	Pass Rate
TestAX25Address	6	Address Field Encoding	Callsign shifting (<<1), SSID masking, reserved bits, C/H bits, last-address flag	100% (6/6)
TestAX25UIFrame	8	UI Frame Assembly & Parse	Flag delimiters, 14-byte address block, 0x03 Control, 0xF0 PID, FCS append, field slicing	100% (8/8)
TestCRC16	5	CRC-16/X-25 Computation	Standard vector 123456789 -> 0x906E, empty payload, single-byte vector, 0xF0B8 residue	100% (5/5)
TestAX25SpecificationEdgeCases	10	Profile Enforcement	Callsign length limits (1-6), empty callsign, lowercase invariant, SSID 0-15 bounds, payload 4078 cap	100% (10/10)
TOTAL PHASE 1	29	Comprehensive Core Suite	Pure Python standard library implementation with zero external runtime dependencies	100% (29/29)

Phase 1 Verification Methodology & Evidence Matrices

Detailed validation against authoritative telecommunication and amateur satellite standards.

1. CRC-16/X-25 Authoritative Vector Matrix

Test Vector / Input Dataset	Payload Size	Expected FCS (Hex)	Calculated FCS (Hex)	Residual Constant Check	Status
ASCII '123456789'	9 bytes	0x906E	0x906E	0xF0B8 (Verified Standard Residual)	PASS
Null / Empty Payload: b''	0 bytes	0x0000	0x0000	0xF0B8 (Verified Standard Residual)	PASS
Single Byte Vector: b'A' (0x41)	1 byte	0xA3F5	0xA3F5	0xF0B8 (Verified Standard Residual)	PASS
Pangram: 'The quick brown fox...'	43 bytes	0x24C8	0x24C8	0xF0B8 (Verified Standard Residual)	PASS
High-density Ones: 0xFF * 64	64 bytes	0x3D72	0x3D72	0xF0B8 (Verified Standard Residual)	PASS

2. Address Field Encoding & Decoding Round-Trip Matrix

Station Identity	SSID	Last Flag	Wire Octet Hex (7 Bytes)	Decoded Output	Round-Trip Status
ORBITL	1	True (Source)	9E A4 84 92 A8 98 63	ORBITL-1 (Last=True)	PASS (100% Match)
GROUND	0	False (Dest)	8E A4 9E AA 9C 88 60	GROUND-0 (Last=False)	PASS (100% Match)
NJ7P	0	False (Dest)	9C 94 6E A0 40 40 60	NJ7P-0 (AX.25 v2.2 Sec 3.12 Spec Vector)	PASS (100% Match)
N7LEM	0	True (Source)	9C 6E 98 8A 9A 40 61	N7LEM-0 (AX.25 v2.2 Sec 3.12 Spec Vector)	PASS (100% Match)
HS0AC	15	True (Source)	90 A6 60 82 86 40 7F	HS0AC-15 (Max SSID 15 boundary)	PASS (100% Match)

3. Strict Specification Boundary Enforcement Matrix

Enforcement Point	Tested Condition	Rejection Mechanism	System Safety Rationale
Callsign String Length	Callsign > 6 characters (e.g. 'SATELLITE-1')	Raises ValueError('Callsign exceeds 6 characters')	Prevents address block buffer overrun and malformed wire framing
Empty Callsign	Zero-length string (e.g. '-0')	Raises ValueError('Callsign cannot be empty')	Ensures all transmitted packets have traceable amateur station ID
SSID Integer Range	SSID < 0 or SSID > 15 (e.g. -1, 16)	Raises ValueError('SSID must be 0-15')	Maintains 4-bit secondary station identifier conformance
Information Field Cap	Payload > 4078 bytes (e.g. 4079 bytes)	Raises ValueError('Information field exceeds 4078 bytes')	Protects Layer 2 memory allocations and MTU constraints

Phase 1 Production Source Code Excerpts (Ground Truth)

Verified excerpts from src/ax25.py illustrating deterministic protocol algorithms.

1. AX.25 UI Frame Construction (src/ax25.py: build_ui_frame)

```
def build_ui_frame(destination: str, source: str, information: bytes) -> bytes:
    information = _require_bytes(information, "information")
    if len(information) > MAX_INFORMATION_BYTES:  # 4078 bytes
        raise ValueError(f"Information field exceeds {MAX_INFORMATION_BYTES} bytes...")
    body = bytearray()
    body.extend(encode_address(destination, last=False))
    body.extend(encode_address(source, last=True))
    body.extend((AX25_UI_CONTROL, AX25_NO_LAYER3_PID))  # 0x03, 0xF0
    body.extend(information)
    fcs = crc16_x25(bytes(body))
    body.extend((fcs & 0xFF, (fcs >> 8) & 0xFF))  # low byte first
    return bytes((AX25_FLAG,)) + bytes(body) + bytes((AX25_FLAG,))
```

2. Address Encoding with Shift and Extension Bits (src/ax25.py: encode_address)

```
def encode_address(address_str: str, last: bool = False) -> bytes:
    callsign, ssid = parse_address(address_str)
    encoded = bytearray(7)
    padded = callsign.ljust(6)
    for i, char in enumerate(padded):
        encoded[i] = ord(char) << 1
    encoded[6] = 0x60 | ((ssid & 0x0F) << 1)
    if last:
        encoded[6] |= 0x01  # Extension bit: 1 marks end of address field
    return bytes(encoded)
```

3. Bitwise CRC-16/X-25 Implementation (src/ax25.py: crc16_x25)

```
def crc16_x25(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            if crc & 0x0001:
                crc = (crc >> 1) ^ 0x8408  # Reverse polynomial of 0x1021
            else:
                crc >>= 1
    return crc ^ 0xFFFF
```

Phase 1 Subsystem Demarcation & Software Acceptance

Interface contract to Phase 2, hardware boundary, and formal sign-off.

Subsystem Interface Contract: Phase 1 to Phase 2

Interface Boundary	Sending Entity	Receiving Entity	Data Format & Structure	Verification Status
Frame Byte Pipeline	build_ui_frame()	AX25Transmitter.encode()	Raw AX.25 frame bytes including 0x7E delimiters and low-byte-first FCS	Verified (Phase 1/2 Handoff)
Frame Parser Interface	HDLCStreamDecoder / Receiver	decode_ui_frame()	Unstuffed, FCS-validated byte sequence passed to Layer 2 parser	Verified (Bit-Accurate)
Error Notification	decode_ui_frame()	Receiver Exception Handler	Explicit ValueError raised on corrupt FCS, bad address, or illegal PID/Control	Verified (Zero False Accept)

Software Implementation vs. Hardware Demarcation

Phase 1 operates purely in the digital protocol domain. All address parsing, byte formatting, and checksum calculations execute natively in Python standard library code. Physical RF parameters (modulation frequency, baud rate, Doppler shift, and transmitter power amplifier control) are intentionally decoupled and scheduled for Phase 5 hardware integration.

Formal Software Acceptance Gate Declaration

PHASE 1 ACCEPTANCE GATE: MET (100% SOFTWARE-VERIFIED)
All 29 automated unit and specification tests in tests/test_ax25_core.py execute cleanly with zero failures or errors.
Conformance with amateur AX.25 v2.2 and ISO/IEC 3309 test vectors has been mathematically demonstrated.
The Phase 1 codebase (src/ax25.py) is locked and verified with SHA-256 hash 8da3d41a47c8211e482270a554cbb04e046867159dadfb789e8944990e30b9f1.